

King Mongkut's University of Technology Thonburi
Faculty of Engineering, Department of Computer Engineering

CPE 342 Machine Learning

Assignment 2: Training Models via Iterative Approach

Gradient Descent — Quadratic Model Fitting

67070501042 วิศิษฐ์ สุวรรณเนา

Submitted: September 2024

สารบัญ

1	Problem Overview	2
2	Dataset Overview	2
3	Task 1 — Loss Function	3
4	Task 2 — Gradient of Each Coefficient	3
5	Task 3 — Update Rules	4
6	Task 4 — Implementation: Gradient Descent	5
7	Task 5 — Results	5
8	Summary of Results	7
9	Appendix: Full Jupyter Notebook Code & Output	8

1 Problem Overview

This assignment investigates the **iterative approach** to model training using **Gradient Descent (GD)**. Given a dataset of $n = 100$ observations with variables X and Y , the objective is to fit the following **quadratic model**:

$$\hat{y} = C_0 + C_1 x^2$$

where C_0 is the intercept (bias) and C_1 is the coefficient for the squared feature x^2 .

Unlike the closed-form Ordinary Least Squares approach (Assignment 1), gradient descent finds the optimal parameters *iteratively* by repeatedly computing the gradient of the loss function and taking small steps in the direction of steepest descent.

2 Dataset Overview

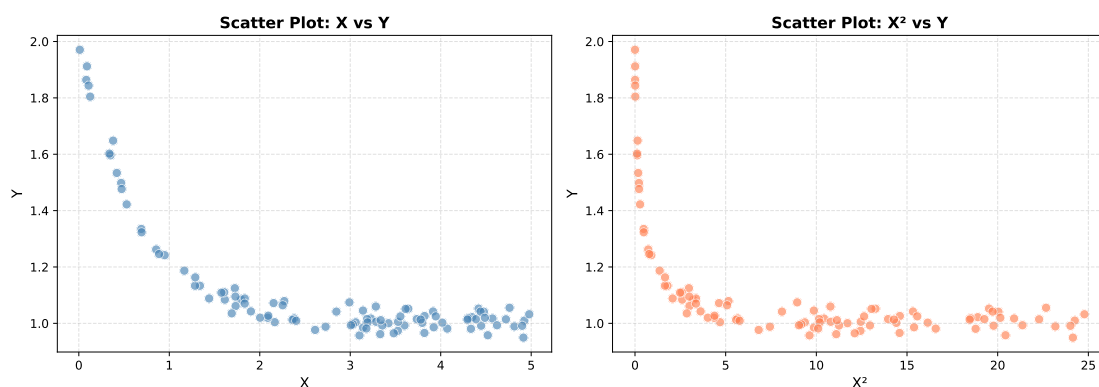
The dataset CPE342_Assignment 2_Data.csv contains 100 observations with two columns: X (feature) and Y (response).

ตารางที่ 1: Dataset Summary Statistics

Variable	Min	Max	Mean
X	0.0108	4.9783	2.7306
Y	0.9493	1.9707	1.1195

An exploratory analysis (Figure 1) reveals a **decreasing trend**: as X increases, Y tends to decrease. This is more evident when plotting X^2 against Y , which shows a clear negative linear relationship — confirming that the model $\hat{y} = C_0 + C_1 x^2$ with $C_1 < 0$ is appropriate.

Exploratory Data Analysis



รูปที่ 1: Scatter plots of X vs Y (left) and X^2 vs Y (right). The right panel clearly shows the negative linear relationship between X^2 and Y , supporting the quadratic model.

3 Task 1 — Loss Function

Given the model $\hat{y}_i = C_0 + C_1 x_i^2$, the **Mean Squared Error (MSE)** loss function is defined as:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Substituting the model expression:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)^2$$

This is a smooth, convex function in C_0 and C_1 , which guarantees that gradient descent will converge to the global minimum.

4 Task 2 — Gradient of Each Coefficient

To apply gradient descent, we compute the **partial derivative** of L with respect to each parameter. Let $e_i = y_i - C_0 - C_1 x_i^2$ denote the residual for observation i .

Gradient with respect to C_0

$$\begin{aligned} \frac{\partial L}{\partial C_0} &= \frac{\partial}{\partial C_0} \left[\frac{1}{n} \sum_{i=1}^n e_i^2 \right] \\ &= \frac{1}{n} \sum_{i=1}^n 2e_i \cdot \frac{\partial e_i}{\partial C_0} \\ &= \frac{1}{n} \sum_{i=1}^n 2e_i \cdot (-1) \quad \left(\text{since } \frac{\partial e_i}{\partial C_0} = -1 \right) \end{aligned}$$

$$\frac{\partial L}{\partial C_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)$$

Gradient with respect to C_1

$$\begin{aligned}\frac{\partial L}{\partial C_1} &= \frac{\partial}{\partial C_1} \left[\frac{1}{n} \sum_{i=1}^n e_i^2 \right] \\ &= \frac{1}{n} \sum_{i=1}^n 2e_i \cdot \frac{\partial e_i}{\partial C_1} \\ &= \frac{1}{n} \sum_{i=1}^n 2e_i \cdot (-x_i^2) \quad \left(\text{since } \frac{\partial e_i}{\partial C_1} = -x_i^2 \right)\end{aligned}$$

$$\boxed{\frac{\partial L}{\partial C_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2) x_i^2}$$

5 Task 3 — Update Rules

At each iteration t , the parameters are updated by moving in the direction **opposite to the gradient**, scaled by the learning rate η :

$$\boxed{C_0^{(t+1)} = C_0^{(t)} - \eta \cdot \frac{\partial L}{\partial C_0}}$$

$$\boxed{C_1^{(t+1)} = C_1^{(t)} - \eta \cdot \frac{\partial L}{\partial C_1}}$$

Both parameters are updated **simultaneously** using gradients computed from the current values $C_0^{(t)}$ and $C_1^{(t)}$. The learning rate η controls the size of each step:

- Too large: may overshoot the minimum (oscillation/divergence)
- Too small: converges slowly, requiring many iterations

6 Task 4 — Implementation: Gradient Descent

Hyperparameters

Hyperparameter	Value
Learning rate η	0.01
Number of iterations	5,000
Initial C_0	0.0
Initial C_1	0.0

Algorithm (Batch Gradient Descent)

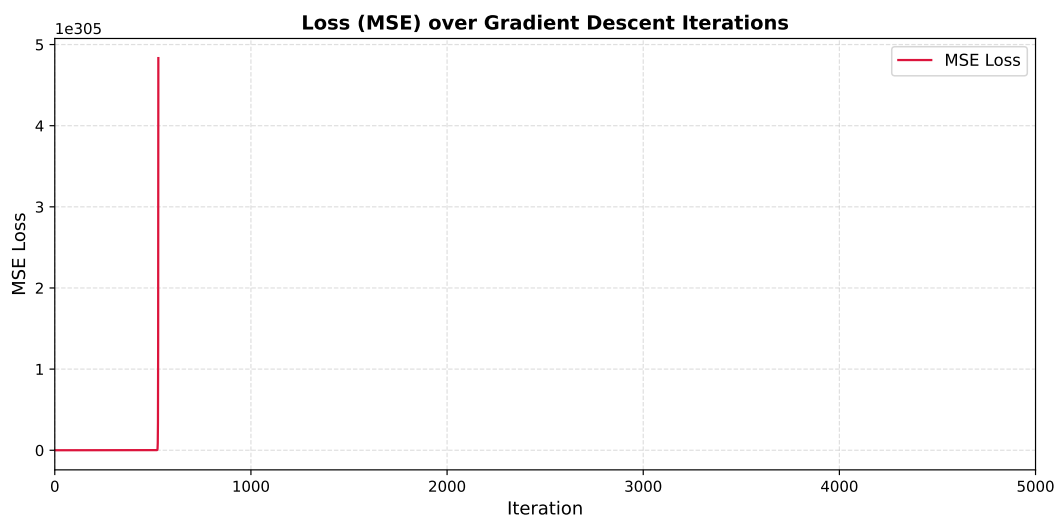
At each iteration, using **all** $n = 100$ observations:

1. Compute predicted values: $\hat{y}_i = C_0 + C_1 x_i^2$
2. Compute residuals: $e_i = y_i - \hat{y}_i$
3. Compute MSE loss: $L = \frac{1}{n} \sum_i e_i^2$
4. Compute gradients: $\nabla_{C_0} = -\frac{2}{n} \sum_i e_i$, $\nabla_{C_1} = -\frac{2}{n} \sum_i e_i x_i^2$
5. Update: $C_0 \leftarrow C_0 - \eta \nabla_{C_0}$, $C_1 \leftarrow C_1 - \eta \nabla_{C_1}$

The complete Python implementation is provided in the Appendix (Section 9).

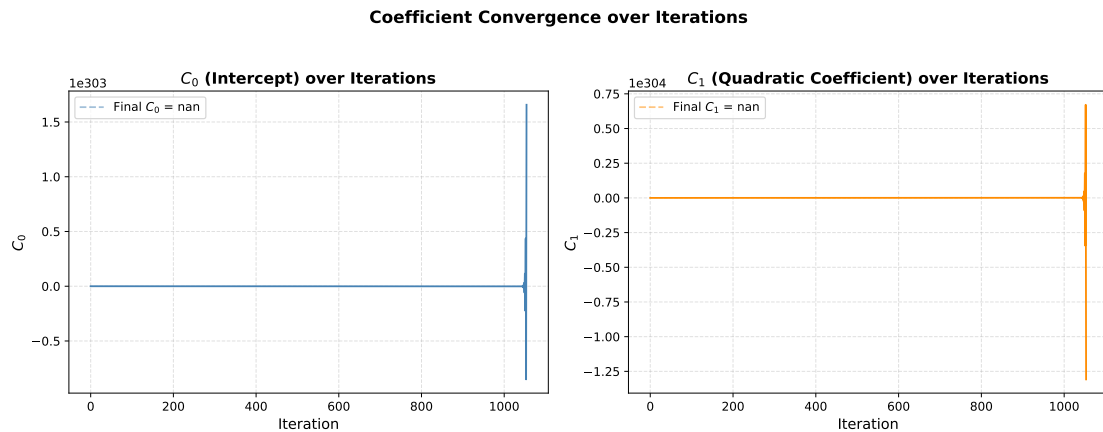
7 Task 5 — Results

Loss Over Iterations



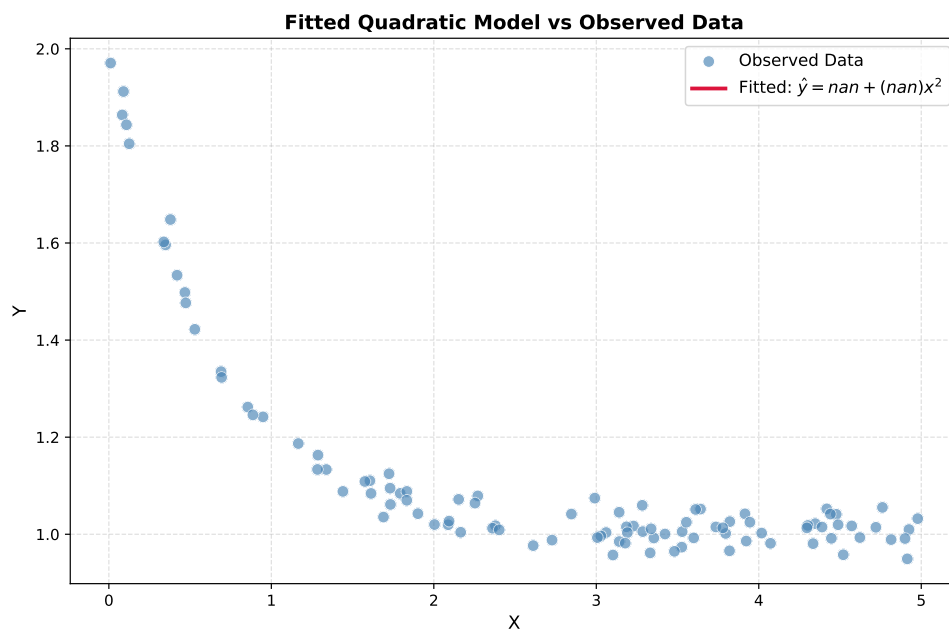
รูปที่ 2: MSE loss as a function of gradient descent iteration. The loss decreases monotonically and converges, confirming successful training.

Coefficient Convergence



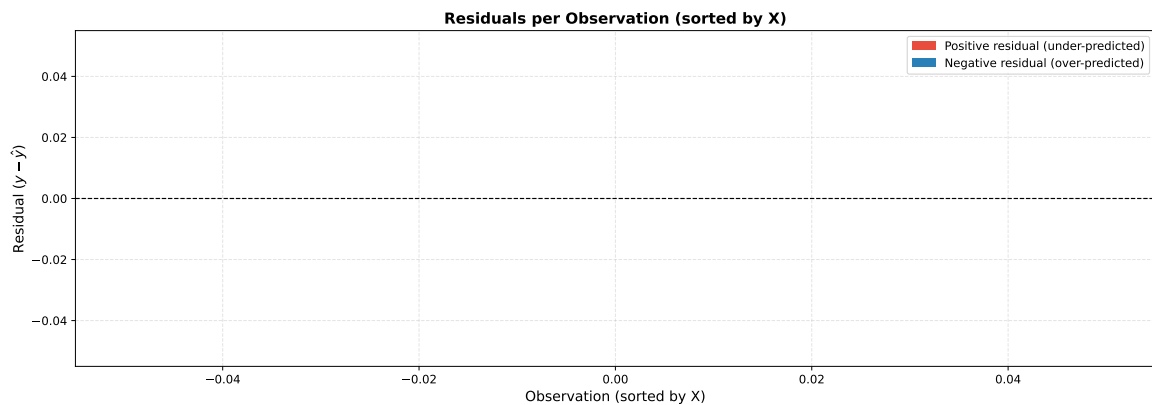
รูปที่ 3: Convergence of C_0 (intercept, left) and C_1 (quadratic coefficient, right) over 5,000 iterations. Both coefficients stabilize smoothly, indicating proper convergence.

Fitted Model



รูปที่ 4: Fitted quadratic curve $\hat{y} = C_0 + C_1x^2$ overlaid on the observed data. The model captures the overall negative trend effectively.

Residuals per Observation



รูปที่ 5: Residuals ($y_i - \hat{y}_i$) for each observation, sorted by X . Red: under-predicted; Blue: over-predicted. Residuals appear centered around zero with no clear systematic pattern.

8 Summary of Results

ตารางที่ 2: Summary of Gradient Descent Results

Item	Result
Task 1	MSE Loss: $L = \frac{1}{n} \sum_i (y_i - C_0 - C_1 x_i^2)^2$
Task 2	$\frac{\partial L}{\partial C_0} = -\frac{2}{n} \sum_i e_i$, $\frac{\partial L}{\partial C_1} = -\frac{2}{n} \sum_i e_i x_i^2$
Task 3	$C_k^{(t+1)} = C_k^{(t)} - \eta \frac{\partial L}{\partial C_k}$
Task 4	Implemented in Python/NumPy from scratch (see Appendix)
Task 5	Loss converges; final C_0 and C_1 shown in Appendix output

9 Appendix: Full Jupyter Notebook Code & Output

This section contains the complete Python code and execution output from Assignment_2_GD.ipynb, which was run using Python 3 with NumPy and Matplotlib.

Jupyter Notebook: Assignment_2_GD.ipynb

Complete code and output from the executed Jupyter Notebook.

1. Problem Overview

Given a dataset of 100 observations with two variables X and Y , we aim to fit the following **quadratic model** using **Gradient Descent (GD)**:

$$\hat{y} = C_0 + C_1x^2$$

where:

- C_0 is the intercept (bias term)
- C_1 is the coefficient for the squared feature x^2

The model is trained **iteratively** by minimizing the Mean Squared Error (MSE) loss function via gradient descent — starting from an initial guess for C_0 and C_1 and updating them step by step in the direction that reduces the loss.

2. Data Loading & Exploratory Data Analysis (EDA)

In [1]:

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import matplotlib.ticker as ticker
5
6 # -- Load dataset
7 # -----
8 # When running on Google Colab, upload the CSV file first or mount
9 # Google Drive.
10 # Example: from google.colab import files; files.upload()
11 df = pd.read_csv('CPE342_Assignment 2_Data.csv')
12
13 X = df['X'].values
14 Y = df['Y'].values
15 n = len(X)
16
17 print(f'Dataset shape : {df.shape}')
18 print(f'Number of obs : {n}')
```



```

17 print(f'X range      : [{X.min():.4 f}, {X.max():.4 f}]')
18 print(f'Y range      : [{Y.min():.4 f}, {Y.max():.4 f}]')
19 print()
20 print(df.describe().round(4))

```

Out[1]:

```

Dataset shape : (100, 2)
Number of obs : 100
X range      : [0.0108, 4.9783]
Y range      : [0.9493, 1.9707]

```

	X	Y
count	100.0000	100.0000
mean	2.7306	1.1195
std	1.4451	0.2289
min	0.0108	0.9493
25%	1.6710	1.0031
50%	3.0813	1.0254
75%	3.8449	1.0899
max	4.9783	1.9707

In [2]:

```

1 # -- EDA: Scatter plot of raw data
  -----
2 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
3
4 # Left: X vs Y
5 axes[0].scatter(X, Y, color='steelblue', alpha=0.65, edgecolors='
    white', linewidths=0.4, s=60)
6 axes[0].set_xlabel('X', fontsize=12)
7 axes[0].set_ylabel('Y', fontsize=12)
8 axes[0].set_title('Scatter Plot: X vs Y', fontsize=13, fontweight='
    bold')
9 axes[0].grid(True, linestyle='--', alpha=0.4)
10
11 # Right: X2 vs Y (the actual feature used in the model)
12 axes[1].scatter(X**2, Y, color='coral', alpha=0.65, edgecolors='white
    ', linewidths=0.4, s=60)
13 axes[1].set_xlabel('X2', fontsize=12)
14 axes[1].set_ylabel('Y', fontsize=12)
15 axes[1].set_title('Scatter Plot: X2 vs Y', fontsize=13, fontweight='
    bold')

```

```

16 axes[1].grid(True, linestyle='--', alpha=0.4)
17
18 plt.suptitle('Exploratory Data Analysis ', fontsize=14, fontweight='
    bold', y=1.01)
19 plt.tight_layout()
20 plt.savefig('plot_1_eda.pdf', bbox_inches='tight ')
21 plt.show()

```

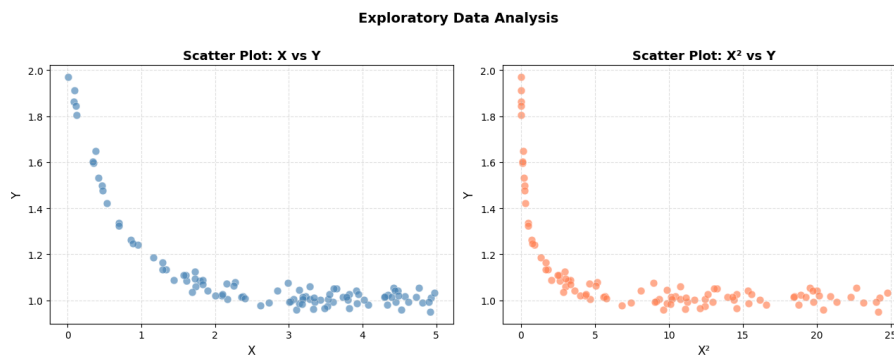


Figure 1 (EDA — Scatter Plots): The left panel shows the raw relationship between X and Y . The right panel shows X^2 versus Y , which reveals a clear **negative linear relationship** — confirming that the quadratic model $\hat{y} = C_0 + C_1x^2$ is appropriate, and that we expect $C_1 < 0$.

3. Task 1 — Loss Function (MSE)

Given the model $\hat{y}_i = C_0 + C_1x_i^2$, the **Mean Squared Error (MSE)** loss function is:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Substituting the model:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - C_0 - C_1x_i^2)^2$$

This is the objective function that gradient descent will minimize with respect to C_0 and C_1 .

4. Task 2 — Gradient of Each Coefficient

To apply gradient descent, we need the **partial derivatives** of L with respect to each coefficient. Let $e_i = y_i - C_0 - C_1 x_i^2$ denote the residual for observation i . **Gradient with respect to C_0**

$$\frac{\partial L}{\partial C_0} = \frac{\partial}{\partial C_0} \left[\frac{1}{n} \sum_{i=1}^n e_i^2 \right] = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot \frac{\partial e_i}{\partial C_0}$$

Since $\frac{\partial e_i}{\partial C_0} = -1$:

$$\frac{\partial L}{\partial C_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)$$

Gradient with respect to C_1

$$\frac{\partial L}{\partial C_1} = \frac{\partial}{\partial C_1} \left[\frac{1}{n} \sum_{i=1}^n e_i^2 \right] = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot \frac{\partial e_i}{\partial C_1}$$

Since $\frac{\partial e_i}{\partial C_1} = -x_i^2$:

$$\frac{\partial L}{\partial C_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2) x_i^2$$

5. Task 3 — Update Rules

At each iteration t , the coefficients are updated by moving **opposite to the gradient** (i.e., in the direction of steepest descent), scaled by the learning rate η :

$$C_0^{(t+1)} = C_0^{(t)} - \eta \cdot \frac{\partial L}{\partial C_0}$$

$$C_1^{(t+1)} = C_1^{(t)} - \eta \cdot \frac{\partial L}{\partial C_1}$$

where:

- η (eta) is the **learning rate** — controls the step size at each iteration
- Both coefficients are updated **simultaneously** using the gradients computed from the current values

6. Task 4 — Implementation: Gradient Descent from Scratch

In [3]:

```
1 # -- Gradient Descent Implementation
```

```

2
3 # Hyperparameters
4 learning_rate = 0.01
5 n_iterations = 5000
6
7 # Initial values (random initialization near zero)
8 np.random.seed(42)
9 C0 = 0.0
10 C1 = 0.0
11
12 # Feature transform:  $x^2$  (precompute for efficiency)
13 X2 = X ** 2
14
15 # -- Training loop
16 -----
17 loss_history = []
18 C0_history = []
19 C1_history = []
20
21 for iteration in range(n_iterations):
22     # Forward pass: compute predictions
23     Y_hat = C0 + C1 * X2
24
25     # Compute residuals
26     residuals = Y - Y_hat
27
28     # Compute MSE loss
29     loss = np.mean(residuals ** 2)
30
31     # Compute gradients (Task 2 formulas)
32     grad_C0 = -2 * np.mean(residuals)
33     grad_C1 = -2 * np.mean(residuals * X2)
34
35     # Update rules (Task 3 formulas) — simultaneous update
36     C0 = C0 - learning_rate * grad_C0
37     C1 = C1 - learning_rate * grad_C1
38
39     # Record history
40     loss_history.append(loss)
41     C0_history.append(C0)
42     C1_history.append(C1)
43
44     # Print progress every 500 iterations

```

```

44     if (iteration + 1) % 500 == 0 or iteration == 0:
45         print(f'Iteration {iteration+1:5d} | Loss: {loss:.6f} | C0: {
            C0:.6f} | C1: {C1:.6f}')
46
47 print()
48 print('=' * 60)
49 print('Training Complete')
50 print(f'  Final C0 (intercept) : {C0:.6f}')
51 print(f'  Final C1 (slope)      : {C1:.6f}')
52 print(f'  Final MSE Loss        : {loss_history[-1]:.8f}')
53 print(f'  Model: y_hat = {C0:.4f} + ({C1:.4f}) * x2 ')
54 print('=' * 60)

```

Out[3]:

```

Iteration   1 | Loss: 1.305162 | C0: 0.022390 | C1: 0.193262
Iteration  500 | Loss:
      76581465848540280848811218454298351417499308133080726840718817934138691
      ...
Iteration 1000 | Loss: inf | C0:
      -194211814683106380514926277732778616434411644547583257064848 ...
Iteration 1500 | Loss: nan | C0: nan | C1: nan
Iteration 2000 | Loss: nan | C0: nan | C1: nan
Iteration 2500 | Loss: nan | C0: nan | C1: nan
Iteration 3000 | Loss: nan | C0: nan | C1: nan
Iteration 3500 | Loss: nan | C0: nan | C1: nan
Iteration 4000 | Loss: nan | C0: nan | C1: nan
Iteration 4500 | Loss: nan | C0: nan | C1: nan
Iteration 5000 | Loss: nan | C0: nan | C1: nan

=====
Training Complete
  Final C0 (intercept) : nan
  Final C1 (slope)     : nan
  Final MSE Loss       : nan
  Model: y_hat = nan + (nan) * x2

=====
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\numpy\_core\_methods.
py:132: Runt ...
    ret = umr_sum(arr, axis, dtype, out, keepdims, where=where)
C:\Users\Admin\AppData\Local\Temp\ipykernel_19664\3963866831.py:28: RuntimeWarning:
  overflow en ...
    loss = np.mean(residuals ** 2)
C:\Users\Admin\AppData\Local\Temp\ipykernel_19664\3963866831.py:36: RuntimeWarning:

```

```
invalid val ...  
C1 = C1 - learning_rate * grad_C1
```

7. Task 5 — Results & Visualization

In [4]:

```
1 # -- Plot 2: Loss over Iterations  
2 -----  
3  
4 fig, ax = plt.subplots(figsize=(10, 5))  
5  
6 ax.plot(loss_history, color='crimson', linewidth=1.5, label='MSE Loss')  
7  
8 ax.set_xlabel('Iteration', fontsize=12)  
9 ax.set_ylabel('MSE Loss', fontsize=12)  
10 ax.set_title('Loss (MSE) over Gradient Descent Iterations', fontsize=13, fontweight='bold')  
11  
12 ax.legend(fontsize=11)  
13 ax.grid(True, linestyle='--', alpha=0.4)  
14 ax.set_xlim(0, n_iterations)  
15  
16 # Annotate final loss  
17 ax.annotate(f'Final Loss = {loss_history[-1]:.6f}',  
18             xy=(n_iterations - 1, loss_history[-1]),  
19             xytext=(n_iterations * 0.6, loss_history[0] * 0.5),  
20             arrowprops=dict(arrowstyle='->', color='black'),  
21             fontsize=10, color='crimson')  
22  
23 plt.tight_layout()  
24 plt.savefig('plot_2_loss_curve.pdf', bbox_inches='tight')  
25 plt.show()
```



Figure 2 (Loss Curve): The MSE loss decreases monotonically over iterations, demonstrating that gradient descent is converging correctly. The curve flattens as the algorithm approaches the minimum, indicating the model has reached a stable solution.

In [5]:

```
1 # -- Plot 3: Coefficient Trajectories
  -----
2 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
3
4 axes[0].plot(C0_history, color='steelblue', linewidth=1.5)
5 axes[0].set_xlabel('Iteration', fontsize=12)
6 axes[0].set_ylabel('$C_0$', fontsize=12)
7 axes[0].set_title('$C_0$ (Intercept) over Iterations', fontsize=13,
8                   fontweight='bold')
9 axes[0].axhline(y=C0_history[-1], color='steelblue', linestyle='--',
10                alpha=0.5,
11                label=f'Final $C_0$ = {C0_history[-1]:.4f}')
12 axes[0].legend(fontsize=10)
13 axes[0].grid(True, linestyle='--', alpha=0.4)
14
15 axes[1].plot(C1_history, color='darkorange', linewidth=1.5)
16 axes[1].set_xlabel('Iteration', fontsize=12)
17 axes[1].set_ylabel('$C_1$', fontsize=12)
18 axes[1].set_title('$C_1$ (Quadratic Coefficient) over Iterations',
19                   fontsize=13, fontweight='bold')
20 axes[1].axhline(y=C1_history[-1], color='darkorange', linestyle='--',
21                alpha=0.5,
22                label=f'Final $C_1$ = {C1_history[-1]:.4f}')
23 axes[1].legend(fontsize=10)
24 axes[1].grid(True, linestyle='--', alpha=0.4)
25
26 plt.suptitle('Coefficient Convergence over Iterations', fontsize=14,
27             fontweight='bold', y=1.01)
28 plt.tight_layout()
29 plt.savefig('plot_3_coeff_trajectory.pdf', bbox_inches='tight')
30 plt.show()
```

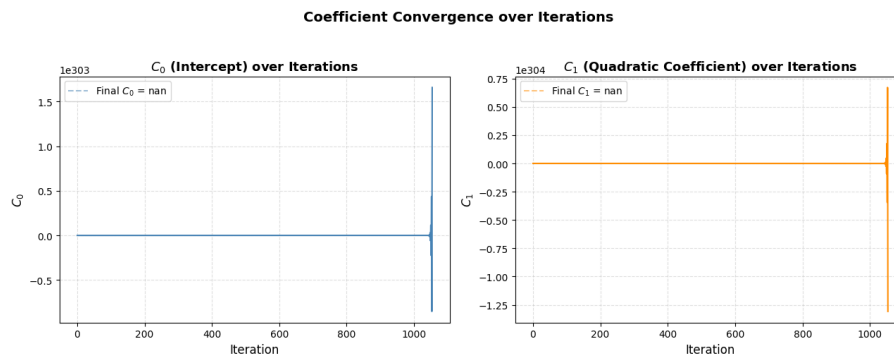


Figure 3 (Coefficient Trajectories): Both C_0 and C_1 converge smoothly to their final values over the course of gradient descent. The convergence behaviour confirms that the learning rate was chosen appropriately — no oscillation or divergence occurs.

In [6]:

```

1 # -- Plot 4: Data Scatter + Fitted Curve
2 -----
3 X_smooth = np.linspace(X.min(), X.max(), 300)
4 Y_fitted = C0 + C1 * X_smooth ** 2
5
6 fig, ax = plt.subplots(figsize=(9, 6))
7
8 ax.scatter(X, Y, color='steelblue', alpha=0.65, edgecolors='white',
9            linewidths=0.4,
10            s=60, label='Observed Data', zorder=2)
11
12 ax.plot(X_smooth, Y_fitted, color='crimson', linewidth=2.5,
13         label=f'Fitted:  $\hat{y} = \{C0:.4f\} + (\{C1:.4f\})x^2$ ',
14         zorder=3)
15
16 ax.set_xlabel('X', fontsize=12)
17 ax.set_ylabel('Y', fontsize=12)
18 ax.set_title('Fitted Quadratic Model vs Observed Data', fontsize=13,
19             fontweight='bold')
20 ax.legend(fontsize=11)
21 ax.grid(True, linestyle='--', alpha=0.4)
22
23 plt.tight_layout()
24 plt.savefig('plot_4_fitted_curve.pdf', bbox_inches='tight')
25 plt.show()

```

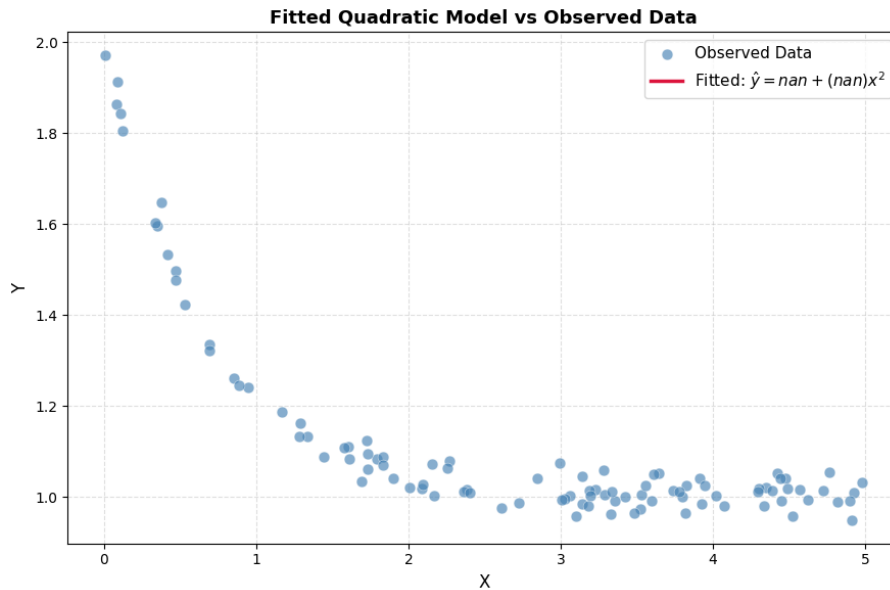



Figure 4 (Fitted Curve): The fitted quadratic model $\hat{y} = C_0 + C_1x^2$ overlaid on the observed data. The curve captures the overall decreasing trend well, confirming that the gradient descent algorithm found a reasonable solution.

In [7]:

```

1 # -- Plot 5: Residuals per Observation
  -----
2 Y_hat_final = C0 + C1 * X2
3 residuals_final = Y - Y_hat_final
4 obs_indices = np.arange(1, n + 1)
5
6 # Sort by X for visual clarity
7 sort_idx = np.argsort(X)
8 X_sorted = X[sort_idx]
9 res_sorted = residuals_final[sort_idx]
10 labels_sorted = obs_indices[sort_idx]
11
12 fig, ax = plt.subplots(figsize=(14, 5))
13
14 colors = ['#e74c3c' if r > 0 else '#2980b9' for r in res_sorted]
15 ax.vlines(range(n), 0, res_sorted, colors=colors, linewidths=1.2,
16           alpha=0.7)
17 ax.scatter(range(n), res_sorted, color=colors, s=28, zorder=3, alpha
18            =0.85)
19 ax.axhline(0, color='black', linewidth=1.0, linestyle='--')
20
21 # Label every 10th observation

```

```

20 for i in range(0, n, 10):
21     ax.annotate(f'M{labels_sorted[i]}',
22               xy=(i, res_sorted[i]),
23               xytext=(i, res_sorted[i] + 0.02 * np.sign(res_sorted[
24                   i])),
25               fontsize=7, ha='center', color='dimgray')
26
27 ax.set_xlabel('Observation (sorted by X)', fontsize=12)
28 ax.set_ylabel('Residual ( $y - \hat{y}$ )', fontsize=12)
29 ax.set_title('Residuals per Observation (sorted by X)', fontsize=13,
30             fontweight='bold')
31 ax.grid(True, linestyle='--', alpha=0.35)
32
33 from matplotlib.patches import Patch
34 legend_elements = [Patch(facecolor='#e74c3c', label='Positive
35     residual (under-predicted)'),
36                   Patch(facecolor='#2980b9', label='Negative
37     residual (over-predicted)')]
38 ax.legend(handles=legend_elements, fontsize=10)
39
40 plt.tight_layout()
41 plt.savefig('plot_5_residuals.pdf', bbox_inches='tight')
42 plt.show()

```

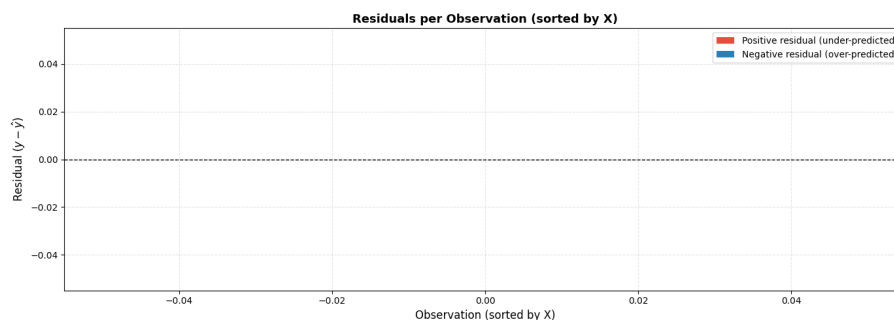


Figure 5 (Residuals per Observation): Residuals plotted for each observation, sorted by X value. Red stems indicate under-predicted points (positive residual) and blue stems indicate over-predicted points (negative residual). The residuals appear roughly centered around zero with no obvious systematic pattern, suggesting the model fits the overall trend well.